

Escola de Programació - Python B1

Projecte Final: Sistema de menjar ràpid

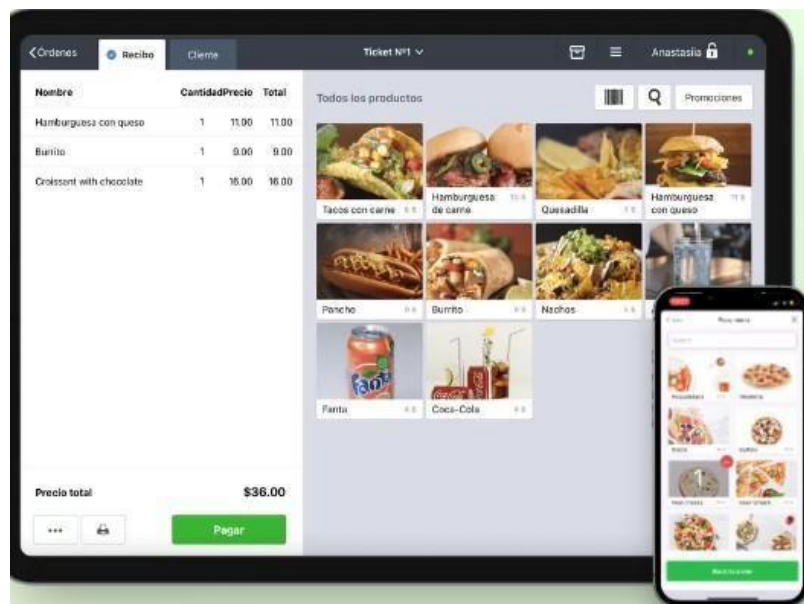
Introducció

Aquest document descriu el projecte final de programació per al nivell B1 de l'Escola de Programació. El projecte consisteix en la implementació d'un sistema de menjar ràpid. Per al seu desenvolupament es posen en pràctica els coneixements adquirits durant el curs.

Criteris d'avaluació

- Demostrar que s'han assimilat correctament els continguts teòrics i que s'apliquen amb precisió en una activitat de caràcter pràctic.
- Raonar totes les respostes de forma analítica i sintètica.
- Estructurar correctament el codi.
- Esquematitzar les respostes mitjançant l'ús d'estructures de control condicionals i de fluxos.
- Capacitat per convertir pseudocodi en una implementació concreta.

Context



Font: Poster. Sistema Per a Restaurants de Menjar Ràpid, 2022

Segons la Wikipedia: "El concepte de menjar ràpid és un estil d'alimentació on el menjar es prepara i serveix per a consumir-se ràpidament en establiments especialitzats".

En aquest exercici abordarem la implementació d'un sistema simple per processar comandes d'un negoci de menjar ràpid. Per això, simularem una base de dades utilitzant diversos fitxers en format CSV, on cadascun tindrà la informació dels caixers, clients i productes. L'objectiu final és crear una aplicació que permeti a un caixer preparar una comanda per a un client, afegint productes i calculant l'import total. És important destacar que farem ús de diversos paquets o components que després s'hauran d'integrar entre si.

Adicionalment, els caixers, clients i productes han de ser representats com a objectes, d'aquesta manera es posaran en pràctica les tècniques del paradigma de la programació orientada a objectes.

Nota: Aquest exercici conté una sèrie de pistes perquè puguis desenvolupar i generar una solució amb tot el que s'ha treballat durant el desenvolupament del curs.

Descripció de les activitats

Implementar un paquet anomenat `products` amb dos mòduls: `food_package.py` i `product.py`, amb la següent estructura:

```
products/  
    _init_.py  
    food_package.py  
    product.py
```

El mòdul `food_package.py` contindrà una classe abstracta anomenada `FoodPackage` amb dues funcions abstractes:

- `def pack(self) -> str`
- `def material(self) -> str`

Aquesta classe ens permet crear un tipus específic de paquet o embolcall depenent del tipus d'aliment a empaquetar. Per exemple:

- Un got de refresc pot ser empaquetat en un paquet tipus got i el material pot ser cartró.
- Una hamburguesa pot ser empaquetada en un paquet tipus embolcall de paper i el material pot ser alumini.

En el mateix mòdul s'hauran d'incloure les implementacions concretes per a cadascuna de les següents classes: `Wrapping`, `Bottle`, `Glass` i `Box`, és a dir, aquestes han d'implementar els mètodes anteriors i retornar un valor. Per exemple, la classe `Wrapping` es pot definir com:

```
class Wrapping(FoodPackage):  
    def pack(self):  
        return "Food Wrap Paper"  
    def material(self):  
        return "Aluminium"
```

El mòdul `product.py` contindrà una classe abstracta anomenada `Product` amb dues funcions abstractes:

- `def type(self) -> str`
- `def foodPackage(self) -> FoodPackage`

Aquesta classe ens permet crear un producte específic i relacionar-lo amb el seu tipus d'embalatge. Per exemple:

- Un producte amb codi de barres **G1** és una soda **Sprite**, el preu de la qual és de **5 euros**, pertany al tipus **Soda** i pot ser empaquetat en un **paquet tipus got**, i el material pot ser **cartró**.
- Un producte amb codi de barres **H1** és una hamburguesa **Bacon**, amb un preu de **15 euros**, pertany al tipus **Hamburger** i pot ser empaquetat en un **paquet tipus embolcall de paper**, i el material pot ser **alumini**.

En el mateix mòdul s'hauran d'incloure les implementacions concretes per a cadascuna de les classes: **Hamburger**, **Soda**, **Drink** i **HappyMeal**. És a dir, de manera similar al mòdul anterior, aquestes han d'implementar els mètodes anteriors i retornar un valor. Per exemple, la classe **Hamburger** es pot definir com:

```
class Hamburger(Product):
```

```

def init(self, id:str, name:str, price:float):
    super().__init__(id,name,price)
def type(self) -> str:
    return "Hamburger"
def foodPackage(self) -> FoodPackage:
    return Wrapping()

```

Implementar un paquet anomenat **users** que conté un mòdul **user.py**, amb la següent estructura:

```

users/
    __init__.py
    user.py

```

El mòdul **user.py** contindrà una classe abstracta anomenada **User**, que té un constructor per defecte per als següents atributs:

```

def __init__(self, dni: str, name: str, age: int),

```

i una funció abstracta:

```

def describe(self)

```

A continuació, en el mateix mòdul s'hauran d'incloure les implementacions concretes per a cadascuna de les classes **Cashier** i **Customer**, és a dir, aquestes han d'implementar els mètodes anteriors i retornar un valor. Addicionalment, aquestes classes es diferencien pels paràmetres que reben als seus constructors, per tant, cal fer ús d'**herència** per inicialitzar el constructor de la classe pare i afegir característiques pròpies a cada classe.

Implementar un paquet anomenat **util** que conté dos mòduls, anomenats **file_manager.py** i **converter.py**, amb la següent estructura:

```

util/
    __init__.py
    file_manager.py
    converter.py

```

El mòdul **file_manager.py** contindrà una classe anomenada **CSVFileManager**, la qual és una **implementació lliure** i ha d'incloure les funcions següents:

- La funció `def read(self)` llegeix un fitxer en format **CSV** i permet exportar-ne el resultat com a un **DataFrame**.
- La funció `def write(self, dataframe)` converteix un **DataFrame** en un fitxer **CSV**. Aquesta és una funció **opcional**, es deixa a criteri de l'estudiant la seva implementació.

Els fitxers en format CSV es troben a la ruta "**data/**", i a continuació es descriu el contingut de cadascun:

- `cashiers.csv`: Informació dels caixers que faran ús del sistema.
- `customers.csv`: Informació dels clients que faran ús del sistema.
- `drinks.csv`: Informació dels diferents tipus de begudes.
- `sodas.csv`: Informació dels diferents tipus de refrescos.
- `hamburgers.csv`: Informació dels diferents tipus d'hamburgueses.
- `happyMeal.csv`: Informació dels diferents tipus de menús infantils (**happy meals**).

El mòdul `converter.py` contindrà una classe anomenada `Converter`, amb una funció abstracta per convertir les files d'un **DataFrame** en instàncies d'objectes. La funció serà:

```
def convert(self, dataframe, *args) -> list
```

Adicionalment, aquesta classe ha d'incloure un mètode que permeti imprimir la informació dels objectes:

```
def print(self, list)
```

En el mateix mòdul s'han d'incloure les implementacions específiques que permetin llegir els fitxers en format **CSV** i convertir-ne les files en objectes de cada classe, utilitzant els paquets `product` i `users`.

Implementar un paquet anomenat `orders` que conté un mòdul `order.py`, amb la següent estructura:

```
orders/  
  __init__.py  
  order.py
```

El mòdul `order.py` contindrà una classe anomenada `Order`, amb un constructor:

```
def __init__(self, cashier: Cashier, customer: Customer)
```

Aquest constructor permet inicialitzar la classe amb les dades del **caixer**, del **client**, i una **llista de productes buida** per defecte.

A més, ha d'incloure tres funcions per:

- Afegir productes: `def add(self, product: Product)`
- Calcular el total de la comanda: `def calculateTotal(self) -> float`
- Mostrar la informació de la comanda en procés: `def show(self)`

Finalment, tindrem una classe principal anomenada `PrepareOrder`, en la qual s'haurà de fer una implementació que permeti **integrar els diferents mòduls** utilitzats per llegir els fitxers CSV i convertir-los en objectes.

La implementació d'aquesta classe és **lliure**, és a dir, no s'indiquen les funcions específiques que ha de contenir, però la funcionalitat ha de permetre:

- Crear un **menú d'opcions** per buscar **clients**, **caixers** i **productes**, i finalment **crear una comanda**.

Es recomana utilitzar **mètodes d'entrada per teclat** per llegir les dades del **DNI del caixer**, del **client**, i de l'**ID dels**

productes.

El funcionament general de l'aplicació seguiria els passos següents:

 Llegir fitxers en format CSV:

- Llegir cada fitxer CSV utilitzant una instància de la classe `CSVFileManager` i cridar el mètode `read()`.

 Convertir a llistes d'objectes:

- **Convertir caixers:** Utilitzar una instància de la classe `XxxConverter`, cridar el mètode `convert()` i imprimir la llista amb el mètode `print()`.
- **Convertir clients:** Igual que amb els caixers.
- **Convertir productes:** Igual que amb els anteriors.

 Preparar una comanda:

1. **Buscar caixer per DNI** (funció creada per l'alumne, retorna una instància de `Cashier`).
2. **Buscar client per DNI** (funció creada per l'alumne, retorna una instància de `Customer`).
3. **Inicialitzar comanda:** Utilitzar una instància de la classe `Order` i inicialitzar-la amb el seu constructor per defecte.
4. **Mostrar productes a vendre:** Funció creada per l'estudiant.
5. **Escollir productes:** Funció creada per l'estudiant.
6. **Afegir productes:** Utilitzar la instància de la classe `Order` del pas c i cridar el mètode `add()`.

Mostrar comanda: Utilitzar la instància de la classe `Order` del pas c i cridar el mètode `show()`. Vegem un exemple dels passos per a dos productes:

1. Llegir arxius en format CSV:

Exemple de codi:

```
df_cashiers = CSVFileManager("ruta/cashier.csv").read()
```

2. Convertir a llistes d'objectes

Convertir caixers: Exemple de codi:

```
converter = XxxConverter()
```

```
cashier_list = converter.convert(df_cashiers)
```



```
converter.print(cashier_list)
```

```
converter.print(cashier_list)
```

Exemple consola:

```
Cashier - Name: Mia, DNI: 5001 , Timetable: 09:00-15:00, Salary: 800.0.  
Cashier - Name: Sinthy, DNI: 5002 , Timetable: 09:00-14:00, Salary: 750.6.  
Cashier - Name: Carles, DNI: 5003 , Timetable: 11:00-17:00, Salary: 655.3.
```

3. Preparar Comanda

a. Cercar caixer per DNI: Funció creada per l'alumne. Exemple de consola:

Introdueix DNI del caixer: 5001

Caixer - Nom: Mia, DNI: 5001 , Horari: 09:00-15:00, Salari: 800.0.

b. Cercar client per DNI: Funció creada per l'alumne. Exemple de consola:

Introdueix DNI del client: 1001

Client - Nom: Mike, DNI: 1001 , Edat: 37, Correu: mike@uoc.edu, Codi Postal: 28009

c. Inicialitzar Comanda: Utilitzar una instància de la classe 'Order' i inicialitzar-la amb el seu constructor per defecte.

Exemple de codi:

```
order = Order(cashier, customer)
```

d. Mostrar productes a vendre: Funció creada per l'alumne. Exemple de consola:

Llista de productes:

Producte - Tipus: Hamburguesa, Nom: Mc Bacon, Id: H1 , Preu: 10.8 , Embolcall: Paper d'alumini per a menjar , Material: Alumini.

Producte - Tipus: Hamburguesa, Nom: Super Delux, Id: H2 , Preu: 15.6 , Embolcall: Paper d'alumini per a menjar , Material: Alumini.

Producte - Tipus: Refresc, Nom: Coca Cola, Id: G1 , Preu: 1.5 , Embolcall: Ampolla, Material: Plàstic.

Producte - Tipus: Refresc, Nom: Seven Up, Id: G3 , Preu: 1.1 , Embolcall: Ampolla, Material: Plàstic.

e. Escollir productes: Funció creada per l'alumne. Exemple de consola:

Introdueix l'identificador del producte: H1

Producte - Tipus: Hamburguesa, Nom: Mc Bacon, Id: H1, Preu: 10.8 , Embolcall: Paper d'alumini per a menjar, Material: Alumini.

Vols afegir un altre producte?: Sí

Introdueix l'identificador del producte: G1

Producte - Tipus: Refresc, Nom: Coca Cola, Id: G1 , Preu: 1.5 , Embolcall: Ampolla, Material: Plàstic.

Vols afegir un altre producte?: No

f. Afegir productes: Utilitzar la instància de la classe 'Order' del pas c i cridar el mètode 'add()'.

4. Mostrar comanda: Utilitzar la instància de la classe `Order` del pas c i cridar el mètode `show()`

5. Exemple de codi:

Hello: Client - Nom: Mike, DNI: 1001 , Edat: 37, Correu: mike@uoc.edu, Codi Postal: 28009

Atès per: Caixer - Nom: Mia, DNI: 5001 , Horari: 09:00-15:00, Salari: 800.0.

Producte 1: Producte - Tipus: Hamburguesa, Nom: Mc Bacon, Id: H1 , Preu: 10.8 , Embolcall: Paper d'alumini per a menjar, Material: Alumini.

Producte 2: Producte - Tipus: Refresc, Nom: Coca Cola, Id: G1 , Preu: 1.5 , Embolcall: Ampolla, Material: Plàstic.

Preu total: 12.3

Nota: Opcionalment es pot implementar la funció `write` de la classe `CSVFileManager` i exportar les comandes a un fitxer nou en format `.csv`, el qual ha de contenir els següents camps per a cada comanda:

- DNI del caixer
- DNI del comprador
- Data i hora de la venda
- Total

Evaluació

La puntuació de cada pregunta és la següent:

- **Pregunta 1:** 100%

Es valorarà la **validesa de la solució** i la **claredat de l'argumentació**.

L'exercici té indicat a l'enunciat el seu pes en la valoració final. Els **criteris d'avaluació** per avaluar aquest exercici són els següents:

Pregunta	No assolit (C-)	Minimament assolit (C+)	Assolit (B)	Assolit de forma excelent (A)
Pregunta 1	La resposta és incorrecta o no ha estat desenvolupada	La resposta és parcialment correcta i està mínimament justificada	La resposta és correcta i descriu alguns passos per arribar a la solució, i està justificada	La resposta és correcta i indica tots els passos per arribar a la solució. Es referencien correctament l'article i els apunts per justificar la resposta

Format

Es suggereix, amb l'objectiu d'estandarditzar el format, que l'activitat segueixi les restriccions següents:

- S'ha de lliurar el codi desenvolupat a través de l'aula Canvas en format .zip.
- Es poden fer diversos lliuraments a la plataforma.
- El codi desenvolupat ha d'incloure comentaris en el codi.
- L'estudiant ha de notificar al Docent Col·laborador amb un missatge un cop consideri que l'exercici està llest per a la qualificació.
- De manera opcional, es pot utilitzar un repositori de GitHub amb el correu de la UOC per pujar-hi la solució. En aquest cas, cal:
 - Posar el repositori en públic perquè el Docent Col·laborador hi pugui accedir.
 - Incloure la URL del repositori en el missatge.
 - Fer servir el repositori de GitHub no exclou l'obligació de lliurar el codi a través de l'aula Canvas.

Lliurament

El lliurament de l'activitat s'haurà de realitzar a la secció: **Activitat Projecte Final de l'aula Canvas**.